DIGITAL SYSTEM DESIGN · WEEKS 11–16

The DSD-8 Processor

Project kickoff — an 8-bit Harvard microprocessor, built and verified in teams of three

Instructor: Asim Samejo · Senior Undergraduate Elective

Today’s agenda

1. **The Machine**
   The frozen spec: registers, instruction set, memory, I/O
2. **Architecture & your choices**
   Block diagram, module ownership, microarchitecture
3. **Verification**
   Golden-trace comparison and the 40-test conformance suite
4. **Deliverables & grading**
   Milestones M1–M6, mark allocation, the oral defence
5. **Submitting through GitHub**
   Classroom, workflow, and the CI “certification” pipeline

PART 1 OF 5

The Machine

What every team is building — byte for byte identical, whichever way you build it

What you are building

A working 8-bit microprocessor in Verilog — instruction memory, data memory, a register file, an ALU, a control unit, and a memory-mapped I/O interface. It must run DSD-8 assembly and match the reference simulator exactly, instruction for instruction.

8-bit
data width

8
general registers, R0–R7

16
opcodes — every one defined

512×16
words of program memory

256×8
bytes of data memory

256
I/O ports, own address space

4
condition flags: Z N C V

0
stack. There are no subroutine calls

On completing this project, you will be able to

- Translate a written spec into working RTL, with no ambiguity left to hide behind
- Partition a system into modules with explicit interface contracts
- Design a control unit as an FSM driving a datapath, and produce the control-signal table that defines it
- Verify a sequential system by trace comparison against a golden model — the technique that transfers directly to industrial processor verification
- Diagnose a fault in a system with thousands of internal states, from evidence rather than guesswork
- Defend, in person and without notes, every design decision you made

What is frozen, what is yours

Frozen — identical for every team	Yours to decide
Instruction set, encodings and semantics	The microarchitecture: single-cycle, multi-cycle or pipelined
Register model, memory sizes, reset state	How the control unit is structured and encoded
Module names and port names	Everything inside each module
The I/O bus protocol and port map	Whether the ALU is one block or several
The execution trace format	Coding style, provided it passes the lint rules
The conformance test suite	Your own additional tests, of which you will need many

THE RULE THAT OVERRIDES EVERYTHING ELSE

“If your processor does not produce the specified trace for the supplied test programs, it does not matter how elegant the inside is.”

Conformance is assessed first and objectively. Build the plainest machine that works, get it conforming, and only then improve it.

The programmer’s model

Registers

Eight general-purpose 8-bit registers, R0–R7. None is hardwired to a constant — **R0 is not special**. All eight are freely read and written by any instruction with a register operand.

Which instructions touch the flags

Exactly eight: ADD SUB AND OR XOR SHL SHR CMP. Every other instruction — including LD, ST, LDI, IN, OUT, JMP, NOP, HLT — leaves all four flags untouched.

Flag	Set to 1 when
Z	the 8-bit result is 0x00
N	bit 7 of the result is 1 (the sign bit)
C	the 9th bit of the internal addition is 1
V	the result is wrong under a signed reading

THE TRAP

After CMP Ra, Rb: **C=1 means no borrow**, i.e. Ra ≥ Rb unsigned. Some architectures define this the other way round — DSD-8 does not.

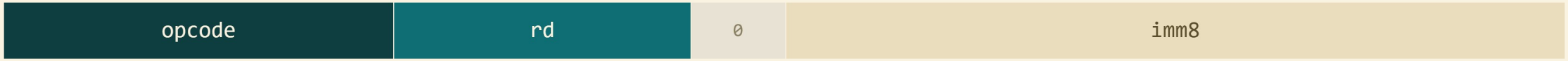
Five formats, one 16-bit word

Bit 15 is on the left. The 4-bit opcode field is always in the same place, so decoding always starts the same way.

R — register to register (ADD, SUB, AND, OR, XOR, SHL, SHR, CMP)



I — immediate (LDI only)



M — memory (LD, ST — rs1 always supplies the address)



J — jump & branch (JMP, B{cc})



P — port I/O (IN, OUT — dir: 0=IN, 1=OUT)



Sixteen opcodes — every one defined

Op	Mnemonic	Does
0x0	NOP	do nothing
0x1	LDI rd,imm	load 8-bit constant
0x2	LD rd,[rs1]	load byte from memory
0x3	ST [rs1],rs2	store byte to memory
0x4	ADD rd,rs1,rs2	add, sets all 4 flags
0x5	SUB rd,rs1,rs2	subtract, sets all 4 flags
0x6	AND rd,rs1,rs2	bitwise AND
0x7	OR rd,rs1,rs2	bitwise OR

Op	Mnemonic	Does
0x8	XOR rd,rs1,rs2	bitwise XOR
0x9	SHL rd,rs1	shift left, MSB→carry
0xA	SHR rd,rs1	logical shift right
0xB	CMP rs1,rs2	compare: flags only, no write
0xC	JMP addr9	unconditional jump
0xD	B{cc} addr9	branch if condition holds
0xE	IN/OUT	read/write an I/O port
0xF	HLT	stop execution

There is no such thing as an illegal opcode in DSD-8 — check your decoder against Appendix A before running a single test program.

Eight branch conditions

Mnemonic	Taken if	Typical use after CMP
BEQ	Z=1	Ra equals Rb
BNE	Z=0	Ra differs from Rb
BCS	C=1	Ra ≥ Rb, unsigned
BCC	C=0	Ra < Rb, unsigned
BMI	N=1	result was negative
BPL	N=0	result positive or zero
BVS	V=1	signed overflow occurred
BVC	V=0	no signed overflow

ON PURPOSE

There is no signed less-than branch. To compare signed values you combine N and V yourself: Ra < Rb signed is true exactly when N ≠ V.

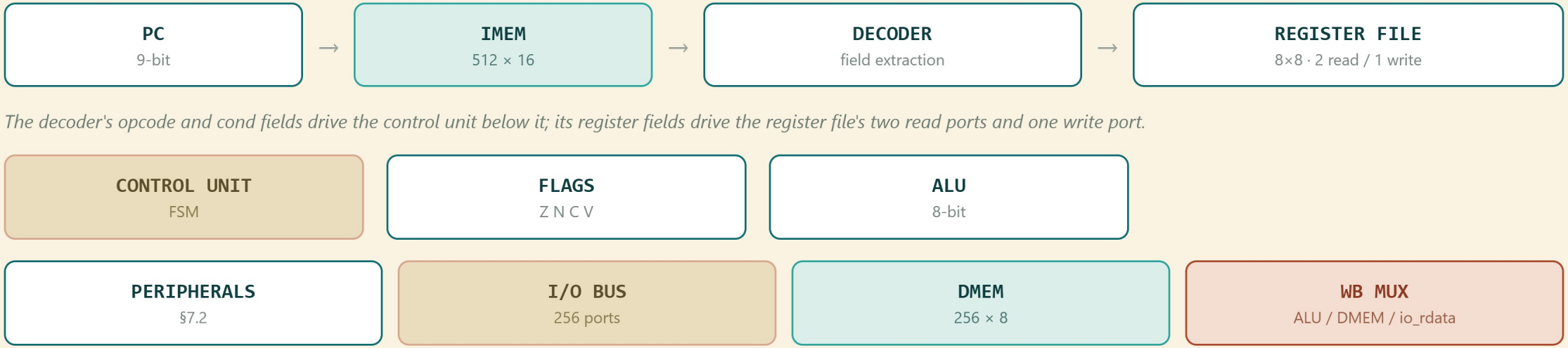
Working that out with the branches you have is a genuinely instructive exercise — and it is why BVS/BVC exist.

PART 2 OF 5

Architecture & Your Choices

The block diagram, who owns what, and the one microarchitecture constraint

The system, in one diagram



The decoder's opcode and cond fields drive the control unit below it; its register fields drive the register file's two read ports and one write port.

The write-back mux selects the ALU result, memory data, or port data and returns it to the register file — and, on a taken branch, the target address returns to the PC.

`white = datapath · tan = control · teal = memory`

Who owns what

Eleven module names are frozen in §5. Split them three ways — each owner is individually examined on their modules at the oral defence.

A

Fetch

`prog_counter`

9-bit PC and next-address selection

`instr_memory`

512×16 read-only, loaded by $readmemh

`instr_decoder`

splits the word into fields; purely combinational

B

Datapath

`register_file`

8×8 registers, two read ports, one write port

`alu`

arithmetic, logic, shifts, and the four flag outputs

`flags_reg`

the architectural Z, N, C, V registers

C

Memory, I/O & control

`data_memory`

256×8 read/write

`control_unit`

the FSM that sequences every instruction

`io_bus`

port address decode and the io_ready handshake

dsd8_top and dsd8_cpu are shared — the whole team integrates them together.

Your choice, with one constraint

The microarchitecture is not specified. Single-cycle, multi-cycle, or pipelined are all acceptable — provided instructions **retire in program order** with exactly one trace line each.

Approach	Cycles / instr	Comment
Multi-cycle FSM	3 to 5	Recommended. Clear states, easy to trace, modest hardware.
Single-cycle	1	Simpler control, but one very long combinational path and a memory that must respond within the cycle.
Pipelined	1 (ideal)	Attempt only after conformance is reached. Hazards must be handled, and retirement order must still be exact.

The control-unit table in the next section assumes the multi-cycle FSM — it maps directly onto the control-unit material from lecture.

The recommended state machine



Instructions that need no memory or I/O access may skip states. A minimal implementation might use only FETCH, EXECUTE and WRITEBACK — three cycles for everything.

THE TWO ROWS THAT CATCH PEOPLE

CMP is the only instruction that sets flags **without** writing a register — if rf_we is high for CMP, you corrupt R[rd]. And ST writes memory but writes no register at all.

The I/O bus: a request/ready handshake

The processor presents a port number and a direction; the addressed peripheral responds, and may take as long as it needs.

Signal	Direction	Width	Meaning
io_port	CPU → bus	8	port number, from IR[7:0]
io_wdata	CPU → bus	8	data to be written by OUT
io_rdata	bus → CPU	8	data returned to IN
io_rd	CPU → bus	1	read request, held until io_ready
io_wr	CPU → bus	1	write request, held until io_ready
io_ready	bus → CPU	1	peripheral is ready to complete the transfer

Assert io_rd/io_wr and hold the bus stable → wait → sample io_ready on a rising edge → deassert and complete. Architectural state may not change while waiting.

WHY BOTHER, WHEN NOTHING NEEDS IT YET?

Test group T5 attaches a deliberately slow peripheral at port 0x40 (SLOW_ECHO) that holds io_ready low for 0–7 random cycles. A processor that ignores the handshake passes every other test and fails exactly that one.

PART 3 OF 5

Verification

How “it works” becomes something you can actually prove

How you will actually find your bugs

A processor has more reachable states than you can ever enumerate, so it cannot be verified by inspecting waveforms. Instead you run the same program on your processor and on a **golden reference simulator**, and compare the two trace files. The first differing line tells you which instruction broke — and what it did wrong.



first differing line = the bug

The supplied harness (tb_dsd8.v) does this wiring for you: it instantiates your dsd8_cpu, loads the hex image, runs until halted or a cycle limit, and writes the trace. You do not modify it.

Ten fields, one line per retired instruction

#	Field	Format	Contents
1	sequence	%04d	instruction count, starting at 0001
2	pc	%03x	address of the instruction that just retired
3	ir	%04x	the instruction word
4	rf_we	%01d	1 if a register was written, else 0
5	rf_addr	%01d	which register, else 0
6	rf_data	%02x	the value written, else 00
7	mem_we	%01d	1 if data memory was written, else 0
8	mem_addr	%02x	the address written, else 00
9	mem_data	%02x	the byte written, else 00
10	flags	%01d%01d%01d%01d	Z, N, C, V after the instruction completed

```
0001 000 120D 1 1 0D 0 00 00 0000  ; LDI R1,13
0002 001 1407 1 2 07 0 00 00 0000  ; LDI R2,7
0003 002 1600 1 3 00 0 00 00 0000  ; LDI R3,0
```

STALE FIELDS FAIL TOO

Fields 5, 6, 8 and 9 shall be zero whenever the matching write-enable is zero. Leaving stale data in these fields fails the comparison even when the architectural behaviour is correct.

Forty programs, six groups, no partial credit

Each test is a numbered assembly file with a stored golden trace. Your score is simply how many you reproduce exactly.

Group	Tests	Focus	Count
T1	T01–T08	data movement and immediates	8
T2	T09–T14	data memory	6
T3	T15–T22	ALU results and flags	8
T4	T23–T30	control flow	8
T5	T31–T35	input and output	5
T6	T36–T40	integration programs	5

RECOMMENDED ORDER OF ATTACK

Do not chase tests in numerical order. Get **T05** (NOP) passing first — it proves fetch, the PC, and the trace all work. Then **T01**, which proves register writeback. Then **T23** (JMP), which proves the branch path. Teams that begin with ADD typically spend two days not knowing whether the bug is in the ALU, the decoder, or the trace.

Conformance is necessary, not sufficient

Passing the 40-program suite proves your processor works on forty programs. It will not help you when T36 fails for a reason buried in the ALU.

Required, beyond the suite

A **self-checking unit testbench** for every module in §5.

A **constrained-random ALU test** comparing at least 10,000 operand pairs against a behavioural reference, covering all eight ALU operations.

A **written verification report**: what has been verified, what has not, and why the remaining risk is acceptable.

Coverage, not assertion
The ALU test must demonstrate that the corner cases 0x00, 0x01, 0x7F, 0x80, 0xFF occur as both operands — reported as coverage evidence, not merely claimed.

BUG LOG
Log every bug: what it was, how it was found (unit test, conformance diff, review, inspection), how long it took. Year over year, bugs found by unit tests cost minutes; bugs found at integration cost days.

PART 4 OF 5

Deliverables & Grading

Milestones M1–M6, how marks are allocated, and the individual oral defence

Six milestones, weeks 11 through 16

M1

End W11 · 6%

ownership agreed; block diagram + microarchitecture; control table; empty CI regression

M2

End W12 · 6%

alu, register_file, instr_decoder complete with unit testbenches; ALU random test running

M3

End W13 · 6%

core integrated and fetching; T05, T01, T23 passing; trace format correct

M4

End W14 · 6%

groups T1–T4 passing (30 of 40); bug log populated

M5

End W15 · 6%

all 40 tests passing; verification report drafted; partner design review done

M6

Week 16 · 10%

final repository, report, demonstration, individual oral defence

Every milestone deadline is marked by a git tag (§§12–13) — grading always reads the tagged commit, not whatever is on main at the moment someone looks.

How the 100% breaks down

Component	Weight	How it is judged
Conformance	30%	Objectively: the fraction of the 40 tests reproduced exactly. No partial credit per test.
Milestones M1–M5	20%	On time and complete, as tabulated on the previous slide.
Verification quality	20%	Unit testbenches, the random ALU test, coverage evidence, the closure argument in the verification report, and the bug log.
Design quality & documentation	15%	Lint-clean RTL; no inferred latches; readable hierarchy; the design-decision record; the final report.
Individual oral defence	15%	Each member examined separately on the modules they own and on the system as a whole.

THE DEFENCE IS A MULTIPLIER, NOT AN ADDITION

Your share of the team marks is **scaled** by your individual oral defence: 1.0 for an authoritative performance, down to 0.4 for a student who cannot trace a signal through their own module. This is announced now, in week 11, so nobody is surprised in week 16.

Ten minutes, no notes, per member

Each member is examined separately, on the modules they own and on the system as a whole. Sample questions are published in advance — so you prepare by understanding, not by memorising.

- Point to the logic that produces flags_we. Walk from the instruction word to that signal.
- What happens if io_ready goes low for two hundred cycles? Show me where your design waits.
- Why does CMP not write a register? What did you have to do differently to achieve that?
- Your trace showed the right value one instruction late. How did you find that, and what was it?
- I want to add an ADDI instruction. Do not write code — tell me every module you would touch and what would break.
- Show me the coverage bin that proves you tested signed overflow, and tell me what it does not prove.

PART 5 OF 5

Submitting Through GitHub

Classroom, everyday workflow, and a CI pipeline that certifies your code before we ever open it

Distributing repos with GitHub Classroom

Teams of three, each with their own private repo seeded from one starter template — set up once per offering.

1. Create a Classroom for the course and connect it to your GitHub organization.
2. Import the DSD-8 starter repository as the assignment template.
3. Create a **group assignment**, team size 3 — students form or are assigned to teams, each gets one private repo.
4. Turn on branch protection on main: require a pull request, require the CI checks to pass.
5. Each team edits its own CODEOWNERS to map A/B/C module ownership to their GitHub usernames.
6. Share the invite link and the M1–M6 milestone calendar.

Branches for work, tags for grading

Everyday workflow

1. Branch per module or feature — e.g. alu-shift-fix.
2. Open a pull request into main.
3. CI runs lint + unit tests (always) and conformance (informational).
4. The module's owner approves — enforced by CODEOWNERS.
5. Merge.

At each milestone deadline

The team tags the graded commit: git tag m3 && git push --tags (then m4, m5, final).

Grading always reads the **tagged commit** — never whatever happens to be on main at the moment someone looks.

Pushing a milestone tag flips the conformance job from informational to **authoritative**.

A repository changes continuously; a deadline needs a fixed point. The tag is that fixed point — fair to grade against, and unambiguous for everyone.

Three CI jobs certify every push

Free, scriptable tools — Verilator and Icarus Verilog — run on GitHub's own runners. No license, no self-hosted hardware. They certify what you push; EDA Playground and VS Code stay your day-to-day editors.

1 · Lint

`verilator --lint-only -Wall`

Catches inferred latches and width mismatches — a named criterion in Design quality (15%).

Always runs. Always gates.

2 · Unit tests

`iverilog + vvp per module`

Runs every module's required self-checking testbench from §5, including the random ALU test.

Always runs. Always gates.

3 · Conformance

`tb_dsd8.v vs. all 40 .hex/.trace pairs`

Runs the supplied harness against every stored golden trace and diffs the result.

Informational on ordinary pushes · authoritative on milestone tags.

Results appear as a GitHub Actions job summary and an automatic comment on the pull request — the same evidence a grader reads, visible to the team the moment they push.

Ownership the platform enforces, not the honour system

```
# rtl ownership mirrors §4.2 of the spec
/rtl/prog_counter.v    @memberA
/rtl/instr_memory.v    @memberA
/rtl/instr_decoder.v   @memberA
/rtl/register_file.v   @memberB
/rtl/alu.v              @memberB
/rtl/flags_reg.v      @memberB
/rtl/data_memory.v    @memberC
/rtl/control_unit.v    @memberC
/rtl/io_bus.v         @memberC
/rtl/dsd8_cpu.v       @memberA @memberB @memberC
/rtl/dsd8_top.v       @memberA @memberB @memberC
```

With branch protection set to **require review from code owners**, GitHub itself blocks a merge until the named owner approves a change to their file. Individual accountability is structural, not a promise — and it lines up exactly with who gets examined on what at the oral defence.

Certify before you push, not after

The starter repo ships a Makefile and a git hook so the same checks CI runs, run on your laptop first.

```
make lint       # verilator --lint-only
make test       # every unit testbench
make conformance # all 40 programs
make all        # lint + test + conformance
```

The pre-push hook

Installed once with scripts/install_hooks.sh. Every git push runs make lint && make test first, and blocks the push on failure.

CI re-runs the exact same commands on GitHub — so nothing depends on remembering to run them yourself.

"Certify the code, then push the code" is not a slogan here — it is the literal order the Makefile and the hook run in.

Eight failure modes this design reliably produces

Collected from Appendix C. Each entry names the symptom — because that is what you will actually see.

Trace shifted by one instruction
retire_* sampled a cycle early or late

ST writes the wrong byte or the wrong address
Format M routing: address from rs1, data from rd

Flags field lags one instruction behind
trace must show flags *after* the instruction completes

Everything works until test T34
the io_ready handshake is being ignored

BCS/BCC branch the wrong way after CMP
the carry convention — C=1 means *no borrow*, i.e. ≥

Reads of data memory return x
data memory was not cleared on reset

Register corrupted after a comparison
rf_we asserted for CMP; CMP writes nothing

SHR of 0x80 gives 0xC0
that is arithmetic shift; DSD-8's SHR is logical — always zero into bit 7

START THIS WEEK

1 Form your team of three and agree module ownership — in writing.

2 Accept the GitHub Classroom invite and clone your repo.

3 Read the spec end to end once, before writing any RTL.

4 Build the trace output before you build the ALU. Get NOP passing before ADD.

Questions? Now is the time — before the first line of RTL.

A closing note on method

"The processor is not the hard part. Sixteen opcodes and eight registers is a small machine, and a careful team will have it fetching within a week. What separates the projects at the end of the semester is whether you can *show* that it is right — and whether, when it is not, you can find out why from evidence instead of staring at the code."